IT-TAGE 2026 · FRANKFURT · 10 DECEMBER 2026

Aspire in Practice

From local development to tests and deployment.

Sven Hennessen · vensas GmbH

vensas

About me



- **Almost 20 years** of software development and architecture
- **Co-founder and co-managing director** of vensas GmbH, founded in 2023
- Consulting and development of **business software** for medium-sized companies
- Away from the computer — **hiking**, **cycling** and **snowboarding**

Sven Hennessen
vensas GmbH

From Tye to Aspire

2019
Project Tye
A Microsoft experiment. A CLI that starts all services.

→

2023
.NET Aspire preview
Build 2023. A tool for the local developer.

→

2024
Aspire GA
May 2024, on .NET 8. A stable API.

→

2025
Aspire on its own
Not tied to .NET releases. Its own version numbers.

This talk uses Aspire 13.6 on .NET 10.

LOCAL DEVELOPMENT

The demo app: the cookie jar

A sensor takes cookies from the office cookie jar. The page shows the fill level in real time.

Sensor	Python, sends cookie-taken events
Broker	RabbitMQ
Server	ASP.NET Core and EF Core, two replicas
Database	PostgreSQL
Frontend	React and Vite, Server-Sent Events



LOCAL DEVELOPMENT

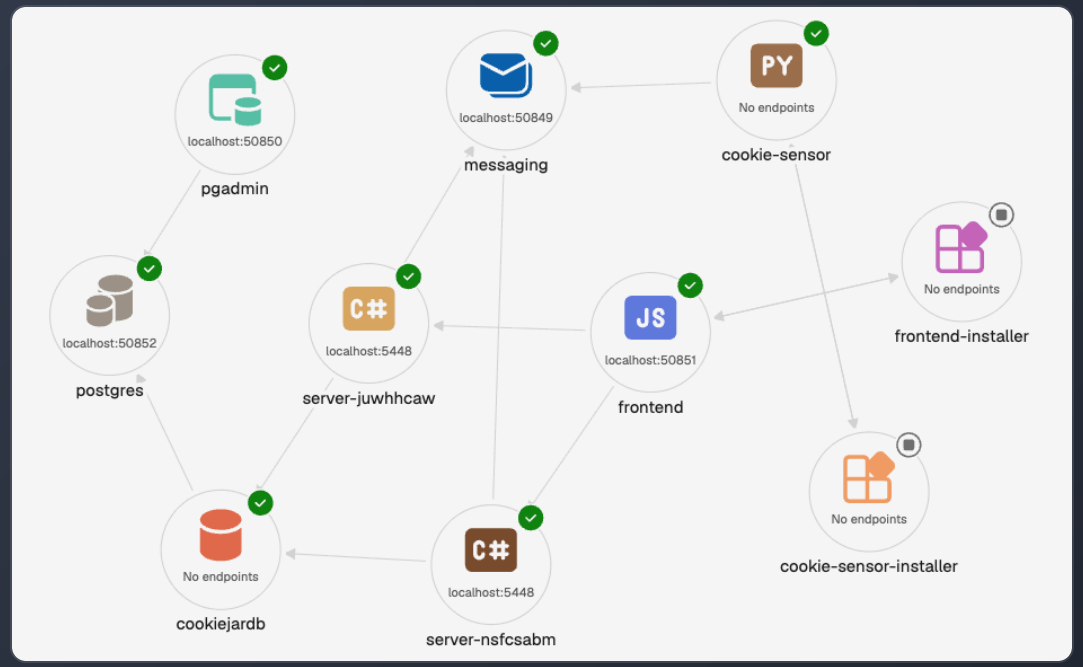
One AppHost for three languages

AppHost.cs

```
var server = builder
  .AddProject<Projects.CookieJar_Server>("server")
  .WithReference(db).WaitFor(db)
  .WithReference(rabbitmq).WaitFor(rabbitmq)
  .WithReplicas(2);

builder.AddPythonApp("cookie-sensor", ...)
  .WithReference(rabbitmq);

builder.AddViteApp("frontend", "../frontend")
  .WithReference(server);
```



`aspire run` starts all of it. `WaitFor` holds the start until the resource is healthy.

LOCAL DEVELOPMENT

Logs, traces and metrics in one dashboard

What you get

- OpenTelemetry from AddServiceDefaults()
- Console and structured logs for each replica
- Traces across the services
- Start, stop and restart for each resource

The two replicas take turns. RabbitMQ gives each event to one of them.



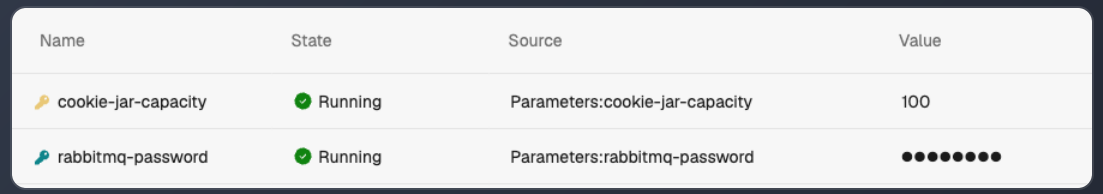
LOCAL DEVELOPMENT

Parameters and secrets

AppHost.cs

```
// A secret has no default value
var password = builder.AddParameter(
    "rabbitmq-password", secret: true);

// Read the configuration, then the default
var capacity = builder.AddParameter(
    "cookie-jar-capacity",
    config["Parameters:cookie-jar-capacity"]
        ?? "100");
```



Name	State	Source	Value
cookie-jar-capacity	Running	Parameters:cookie-jar-capacity	100
rabbitmq-password	Running	Parameters:rabbitmq-password	●●●●●●●●

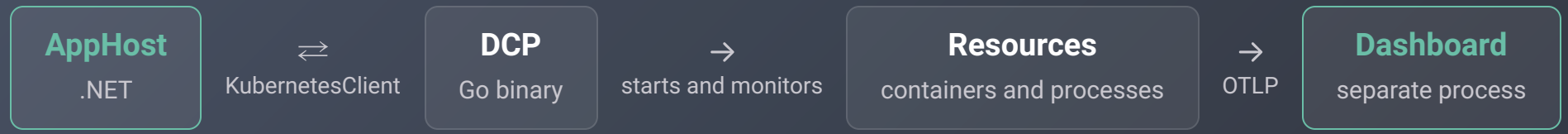
Watch out

AddParameter(name, "100") sets a constant value. Aspire then ignores the configuration. Read the configuration first, as above.

LOCAL DEVELOPMENT

How Aspire works

Runtime architecture The AppHost translates the model. DCP orchestrates the resources.



Annotation model

Each resource is an `IResource` with annotations. `Build()` makes the model real.

WithReference

Gives connection strings and service URLs as environment variables. No ports in the code.

TESTING

Test the real app

Aspire.Hosting.Testing starts your AppHost in the test. The test gets the same services and containers as `aspire run`.

1 Check the model. No containers start.

AppModelTests.cs

```
var builder = await
  DistributedApplicationTestingBuilder
    .CreateAsync<Projects.CookieJar_AppHost>();

var server = builder.Resources
    .Single(r => r.Name == "server");
Assert.Equal(2, server.GetReplicaCount());
```

2 Start the app and send a request.

CookieJarAppFixture.cs

```
App = await builder.BuildAsync(ct);
await App.StartAsync(ct);
await App.ResourceNotifications
    .WaitForResourceHealthyAsync("server", ct);

var http = App.CreateHttpClient("server");
```

Recommendation

Run step 1 in each PR build. It takes less than 1 s. Step 2 starts the containers one time for all tests.

TESTING

Prepare the AppHost for the test

Before `BuildAsync()`, the test can change the app model. Three changes make the test reliable.

CookieJarAppFixture.cs

```
// 1. Give the parameters as arguments. No user secrets are necessary.
var builder = await DistributedApplicationTestingBuilder.CreateAsync<Projects.CookieJar_AppHost>(
    ["Parameters:rabbitmq-password=e2e-test", "Parameters:cookie-jar-capacity=42"]);

// 2. Remove the UI and the random sensor. The test sends the events.
foreach (var r in builder.Resources.Where(r => r is PgAdminContainerResource
             || r.Name.StartsWith("frontend") || r.Name.StartsWith("cookie-sensor")).ToList())
    builder.Resources.Remove(r);

// 3. Remove the data volume. Each run starts with an empty database.
foreach (var mount in postgres.Annotations.OfType<ContainerMountAnnotation>().ToList())
    postgres.Annotations.Remove(mount);
```

The changes apply only to the test. `aspire run` does not change.

TESTING

Two replicas show a bug

The test starts 2 server replicas, as in production. It sends 4 events. Each replica must stream all 4 to its browsers.

CookieJarTests.cs

```
// Start: 100. Each event takes 1 cookie.
var streams = fixture.Replicas
    .Select(ReadLevelStreamAsync).ToList();

for (var i = 0; i < 4; i++)
    await PublishCookieTakenAsync(amount: 1);

foreach (var stream in streams)
    Assert.Equal([100, 99, 98, 97, 96],
        await stream);
```

```
Expected: [100, 99, 98, 97, 96]
Actual:   [100, 99, 98]
```

The bug

The database is correct. But only the replica that got the event tells its browsers.

With 1 instance

The same test passes. Thus the bug stays hidden until production.

TESTING · FROM A REAL PROJECT

A delivery service with two replicas

E2ETestHost.cs

```
// Each test run gets its own containers
foreach (var r in appHost.Resources
             .OfType<ContainerResource>())
    r.Annotations.Add(new ContainerLifetimeAnnotation
        { Lifetime = ContainerLifetime.Session });

// The upstream system is a fake in the test process
appHost.CreateResourceBuilder<ProjectResource>("api")
    .WithEnvironment("UPSTREAM_URL", fakeUpstream.Url);

// Chaos: freeze the storage, then check the behavior
await DockerAsync("pause", storageContainerId);
```

What the tests check

Lock	Only one replica migrates. The cron job runs one time per tick.
Outage	The cache answers. Else 503 with Retry-After.
Runtime	Nightly, about 8 minutes.

Recommendation

Run the E2E tests nightly. An environment variable turns the project on, thus the PR build skips it.

TESTING · FROM A REAL PROJECT

A business application with a web UI

AspireAppFixture.cs

```
appHost.CreateResourceBuilder<ProjectResource>("api")
    // The real external system is not reachable
    .WithEnvironment("EXTERNAL__MASTERDATA__BASEADDRESS",
        "http://127.0.0.1:9")
    // Healthy = the API runs and asks for a login
    .WithHttpHealthCheck("/v1/me", statusCode: 401);

await TestData.SeedAsync(NpgsqlDataSource.Create(
    await app.GetConnectionStringAsync("app-db")));
```

How the team works

- One test for each acceptance criterion
- UI, API and database in one run
- Playwright keeps the trace only after a failure

Recommendation

Make each external system unreachable in the test. Then no test can call it by mistake.

DEPLOYMENT

Docker Compose

AppHost.cs

```
builder.AddDockerComposeEnvironment(
        "docker-compose")
    .ConfigureComposeFile(c =>
        c.Services["server"]
            .DependsOn["postgres"]
            .Condition = "service_healthy");

// aspire publish --output-path ./deploy/output
```

deploy/output/docker-compose/docker-compose.yaml

```
server:
  image: "${SERVER_IMAGE}"
  environment:
    CookieJar__Capacity: "${COOKIE_JAR_CAPACITY}"
    MESSAGING_PASSWORD: "${RABBITMQ_PASSWORD}"
  depends_on:
    postgres:
      condition: "service_healthy"
```

Azure is a first-class target too

`azd up` reads the same model. It is not part of this demo.

Generated by aspire publish

The secrets come from `.env`.

DEPLOYMENT

Kubernetes and Kamal

AppHost.cs

```
builder.AddKubernetesEnvironment("kubernetes");

// aspire publish → deploy/output/helm
// helm upgrade --install cookiejar \
//     ./deploy/output/helm
```

AppHost.cs

```
builder.AddKamalEnvironment("kamal")
    .WithServers(vmIp)
    .WithRegistry(registry, username)
    .WithProxyHostSuffix($"{vmIp}.nip.io");

server.PublishAsKamalService((_, config) =>
    config.Proxy!.Healthcheck =
        new() { Path = "/health" });
```

Kubernetes is a preview

The Helm chart is a start, not the final chart. Plan time to complete it.

Kamal

Deploys containers to plain servers over SSH. It needs no Kubernetes. kamal-deploy.org

DEPLOYMENT

Inside the Kamal package: your own deployment target

1. **A compute environment**
   `IComputeEnvironmentResource`
2. **Pipeline steps**
   `PipelineStepAnnotation`: prepare, publish, deploy
3. **Map the app model**
   Project → app. Container → accessory.
4. **Resolve the values**
   Secret → reference. Endpoint → container name.

deploy/output/kamal/config/deploy.yml

```yaml
service: server
image: cookiejar/server
proxy:
  host: server.192.168.64.5.nip.io
env:
  secret:
    - ConnectionStrings__cookiejardb
accessories:
  postgres:
    image: docker.io/library/postgres:18.3
```

Experimental API

`ASPIREPIPELINES001`

Install

```
dotnet add package Vensas.Aspire.Hosting.Kamal
```

AI

Aspire MCP: the agent sees the running app

The MCP server gives the agent the resources, logs and traces of the running app. Not only the code.

```
aspire agent init     # writes the MCP configuration for Claude Code, Copilot and others
```

1. **Prompt**
   Why does replica 2 not stream the new level?
2. **Logs**
   The agent reads the structured logs of both replicas through MCP.
3. **Answer**
   The cause, with the log lines. You decide on the fix.

```
> Why does replica 2 not stream the new level?

● aspire · list_structured_logs "level/stream"
  server-xyzjuknq  GET /api/cookiejar/level/stream

● aspire · list_structured_logs "level now"
  server-xyzjuknq  Cookie taken, level now 58
  server-khhdbtbw  Cookie taken, level now 49
  server-xyzjuknq  Cookie taken, level now 43
  server-khhdbtbw  Cookie taken, level now 39

The page streams from server-xyzjuknq. RabbitMQ
gives every second event to server-khhdbtbw,
and a replica tells only its own SSE clients.
Thus the page skips 49 and 39. The database is
correct. Fix: a fanout exchange or LISTEN/NOTIFY.
```

TL;DR

1. **Use Aspire for local development now.**
   The full stack with one command, in any language, with OpenTelemetry.
2. **Test with the real AppHost and two replicas.**
   Aspire.Hosting.Testing shows the problems that one instance hides.
3. **Outside Azure, plan time for the deployment.**
   Compose and Helm are a start. Kamal shows that the provider model works.
4. **AI: give the agent the running app with Aspire MCP.**
   Resources, logs and traces are the context. A human decides on the fix.

That's all folks!

THANK YOU

Questions?

Demo project and slides available at https://github.com/vensas/aspire-2026-talk

https://www.linkedin.com/in/svenhennessen/

kontakt@vensas.de

vensas.de

github.com/vensas/aspire-hosting-kamal



Slides & Demo